

Практическое занятие №7: Машинное обучение с `scikit-learn` I.

Подготовка данных для ML, разделение на выборки, первая модель (линейная регрессия или k-NN)

Цели занятия

1. Обучающая:

Сформировать у студентов практические навыки подготовки данных для решения задач машинного обучения: выделение признаков (матрица x) и целевой переменной (y), разделение набора данных на обучающую и тестовую выборки с помощью функции `train_test_split()`.

2. Развивающая:

Научить студентов применять алгоритмы линейной регрессии и k-ближайших соседей (k-NN) для решения задач регрессии и классификации, а также осознанно выбирать между ними в зависимости от характера данных.

3. Воспитательная:

Привить культуру объективной оценки моделей машинного обучения, акцентировать внимание на недопустимости использования тестовых данных в процессе обучения.

Образовательные результаты:

По завершении занятия студент будет уметь:

- Загружать и подготавливать данные для обучения (разделение на признаки и целевую переменную).
- Применять функцию `train_test_split()` для разбиения данных на обучающую и тестовую выборки.
- Обучать модели линейной регрессии и k-NN с помощью библиотеки `scikit-learn`.
- Вычислять и интерпретировать ключевые метрики качества: MSE, MAE, R^2 (для регрессии); accuracy, precision, recall, F1-score (для классификации).
- Сравнивать качество нескольких моделей и выбирать лучшую для конкретной задачи.

Задачи

1. Загрузить встроенный датасет из библиотеки `scikit-learn` (или `seaborn`).
2. Выполнить первичный анализ данных (размерность, типы, пропуски, первые строки).
3. Разделить данные на признаки (x) и целевую переменную (y).
4. Разбить данные на обучающую (80%) и тестовую (20%) выборки с фиксацией `random_state` для воспроизводимости.
5. Обучить модель линейной регрессии (для регрессионной задачи) на обучающей выборке.

6. Обучить модель k-ближайших соседей (для классификационной задачи) на обучающей выборке.
7. Выполнить предсказания на тестовой выборке для обеих моделей.
8. Оценить качество моделей с помощью соответствующих метрик.
9. Сравнить полученные метрики двух моделей и сформулировать вывод о том, какая модель лучше справляется с задачей.
10. Оформить решение в виде Jupyter Notebook с комментариями и выводами.

Теоретические основы

1. Подготовка данных для машинного обучения

Любая задача контролируемого обучения требует выделения из исходного набора данных двух компонентов:

- **Признаки (features)** — входные переменные, на основе которых модель будет делать предсказания. Обозначаются заглавной буквой X и представляют собой матрицу размерности $(n_samples, n_features)$.
- **Целевая переменная (target)** — величина, которую мы хотим предсказать. Обозначается строчной буквой y и представляет собой вектор длины $n_samples$.

```
# Пример загрузки датасета и разделения на X и y
from sklearn.datasets import load_diabetes

data = load_diabetes()
X = data.data      # матрица признаков
y = data.target    # целевая переменная
```

2. Разделение на обучающую и тестовую выборки

Ключевой принцип машинного обучения — модель должна проверяться на данных, которые она **не видела** во время обучения. Это единственный способ реально оценить её способность к обобщению.

Функция `train_test_split()` из модуля `sklearn.model_selection` автоматически разбивает данные на две непересекающиеся части.

```
python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 20% данных — в тестовую выборку
    random_state=42,        # фиксируем для воспроизводимости
    stratify=None           # для классификации — сохраняет пропорции классов
)
```

- **Тестовый размер:** `test_size=0.2` означает, что 20% данных уйдёт на тест, 80% — на обучение. Классическое соотношение — 80/20.
- `random_state` — «замораживает» случайное разбиение, чтобы при повторном запуске кода распределение было идентичным.

- `stratify` — для несбалансированных классификационных задач сохраняет ту же долю каждого класса в тестовой выборке, что и в исходных данных.

3. Линейная регрессия (задача регрессии)

Линейная регрессия моделирует связь между признаками и целевой переменной в виде линейной функции:

$$y = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$

где w_0 — свободный член (intercept), а w_i — коэффициенты при признаках.

В библиотеке `scikit-learn` алгоритм реализован в классе `LinearRegression`. Этот метод быстр в обучении и даёт интерпретируемые коэффициенты, которые показывают вклад каждого признака в предсказание.

```
from sklearn.linear_model import LinearRegression

# Создаём и обучаем модель
model = LinearRegression()
model.fit(X_train, y_train)

# Коэффициенты модели
print("Коэффициенты:", model.coef_)
print("Свободный член:", model.intercept_)

# Предсказание на тестовых данных
y_pred = model.predict(X_test)
```

Метрики качества для регрессии:

Метрика	Формула	Смысл	Чем лучше
MSE (Mean Squared Error)	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Средний квадрат ошибки	меньше
MAE (Mean Absolute Error)	$\frac{1}{n} \sum y_i - \hat{y}_i $	Средняя абсолютная ошибка	меньше
R² (коэф. детерминации)	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	Доля дисперсии, объяснённая моделью	ближе к 1

Типичные диапазоны R^2 зависят от предметной области, но в большинстве практических задач значение выше 0,7–0,8 считается хорошим.

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

4. k-ближайших соседей (k-NN) для классификации

Алгоритм k-ближайших соседей (k-Nearest Neighbors) относится к классу **ленивых алгоритмов**: он не строит явную модель, а просто запоминает все точки обучающей выборки. При классификации нового объекта алгоритм ищет k ближайших (в смысле

евклидова расстояния) соседей в обучающем наборе и присваивает объекту класс, наиболее часто встречающийся среди этих соседей.

Выбор k — ключевой момент: при $k = 1$ модель очень чувствительна к шуму (переобучение), при слишком большом k границы между классами становятся слишком гладкими.

```
from sklearn.neighbors import KNeighborsClassifier

# Создаём и обучаем модель
knn = KNeighborsClassifier(n_neighbors=5)    # k = 5
knn.fit(X_train, y_train)

# Предсказание
y_pred = knn.predict(X_test)
y_prob = knn.predict_proba(X_test)  # вероятности принадлежности к классам
```

Метрики качества для классификации:

Метрика	Формула	Смысл	Чем лучше
Accuracy (точность)	$\frac{TP + TN}{TP + TN + FP + FN}$	Доля правильно классифицированных объектов	выше 0,9
Precision (точность предсказания)	$\frac{TP}{TP + FP}$	Из предсказанных положительных сколько действительно положительных	выше
Recall (полнота)	$\frac{TP}{TP + FN}$	Из реальных положительных сколько модель нашла	выше
F1-score	$2 \cdot \frac{precision \cdot recall}{precision + recall}$	Гармоническое среднее precision и recall	выше

Для многоклассовой классификации часто используют **macro-average** (усреднение по всем классам) или **weighted-average** (усреднение с весами по поддержке классов).

```
from sklearn.metrics import accuracy_score, classification_report

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
print("Точность:", accuracy)
print(report)
```

5. Линейная регрессия или k-NN: что выбрать?

Критерий	Линейная регрессия	k-NN
Тип задачи	Регрессия	Классификация (и регрессия)
Связь признаков	Предполагает линейную зависимость	Моделирует любые нелинейности
Интерпретируемость	Высокая (коэффициенты)	Низкая
Чувствительность к шуму	Низкая	Высокая
Чувствительность к масштабу признаков	Низкая (только при регуляризации)	Критическая (требуется нормализация!)

Размер данных	Эффективен при любом размере	Плохо масштабируется на больших данных
Обучение	Быстрое	Мгновенное (ленивый алгоритм)
Предсказание	Быстрое	Медленное (поиск соседей)

Сравнительный анализ алгоритмов показывает, что для данных с нелинейной структурой k-NN часто оказывается более точным, но при этом требует стандартизации признаков.

Подробный пример (сквозной)

Пример 1: Линейная регрессия на датасете «Диабет»

```
# Шаг 1: Импорт библиотек
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Шаг 2: Загрузка данных
diabetes = load_diabetes()
X = diabetes.data # матрица признаков (442 объекта, 10 признаков)
y = diabetes.target # целевая переменная (прогрессирование диабета)

# Шаг 3: Разделение данных
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Обучающая выборка: {X_train.shape[0]} объектов")
print(f"Тестовая выборка: {X_test.shape[0]} объектов")

# Шаг 4: Обучение модели
model = LinearRegression()
model.fit(X_train, y_train)

# Шаг 5: Предсказание
y_pred = model.predict(X_test)

# Шаг 6: Оценка качества
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("--- Оценка модели ---")
print(f"MSE: {mse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"R²: {r2:.3f}")

# Анализ важности признаков (необязательно)
feature_names = diabetes.feature_names
coefficients = pd.Series(model.coef_, index=feature_names)
print("\n--- Важность признаков ---")
print(coefficients.sort_values(ascending=False))
```

Ожидаемые результаты: MSE ~ 3000, R² около 0.45–0.50 (датасет нелинейный, поэтому линейная модель даёт не очень высокую объяснённую дисперсию).

Пример 2: k-NN классификация на датасете «Ирисы Фишера»

```
# Шаг 1: Импорт библиотек
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Шаг 2: Загрузка данных
iris = load_iris()
X = iris.data
y = iris.target
print("Названия классов:", iris.target_names)
print("Названия признаков:", iris.feature_names)

# Шаг 3: Стандартизация признаков (критична для k-NN!)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Шаг 4: Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Шаг 5: Обучение модели k-NN (k=3)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# Шаг 6: Предсказание
y_pred = knn.predict(X_test)

# Шаг 7: Оценка качества
accuracy = accuracy_score(y_test, y_pred)
print(f"Точность (Accuracy): {accuracy:.3f}")

print("\n--- Детальный отчёт по классам ---")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Шаг 8: Визуализация для разных k
k_values = range(1, 21)
train_scores = []
test_scores = []

for k in k_values:
    knn_tmp = KNeighborsClassifier(n_neighbors=k)
    knn_tmp.fit(X_train, y_train)
    train_scores.append(knn_tmp.score(X_train, y_train))
    test_scores.append(knn_tmp.score(X_test, y_test))

plt.figure(figsize=(10, 6))
plt.plot(k_values, train_scores, label='Обучающая выборка', marker='o')
plt.plot(k_values, test_scores, label='Тестовая выборка', marker='s')
plt.xlabel('k')
plt.ylabel('Точность')
```

```
plt.legend()
plt.title('Зависимость точности от параметра k')
plt.show()

# Находим оптимальное k
best_k = k_values[test_scores.index(max(test_scores))]
print(f"Наилучшее k: {best_k} с точностью {max(test_scores):.3f}")
```

Ожидаемые результаты: при $k=3$ точность ~ 0.97 – 1.0 , график показывает, что обучение и тест сходятся при $k \geq 5$.

Варианты для самостоятельного выполнения (10 вариантов)

Для каждого варианта студент должен:

- Загрузить указанный датасет.
- Выполнить разбиение на признаки и целевую переменную.
- Разделить данные на обучающую (80%) и тестовую (20%) выборки (`random_state = 42`).
- Обучить **линейную регрессию** (для регрессии) или **k-NN** (для классификации) на обучающей выборке.
- Вычислить не менее двух метрик качества и интерпретировать их.
- Попытаться улучшить результат (изменение k для k-NN, подбор масштабирования признаков) и сделать сравнение.
- Оформить отчёт с выводами.

Вариант 1: Диабет (задача регрессии)

- **Датасет:** `load_diabetes()` из `sklearn.datasets`
- **Цель:** прогнозирование прогрессирования диабета по 10 клиническим признакам.
- **Особенность:** попробовать масштабирование признаков (`StandardScaler`) и сравнить качество с моделью без масштабирования.

Вариант 2: Цены на жильё в Калифорнии (задача регрессии)

- **Датасет:** `fetch_california_housing()` из `sklearn.datasets`
- **Цель:** прогнозирование медианной стоимости дома по 8 признакам.
- **Особенность:** сравнить линейную регрессию с `KNeighborsRegressor()` (для этого понадобится дополнительно изучить `KNeighborsRegressor`).

Вариант 3: Рак груди (задача бинарной классификации)

- **Датасет:** `load_breast_cancer()` из `sklearn.datasets`
- **Цель:** классификация опухоли как злокачественной (`malignant`) или доброкачественной (`benign`).
- **Особенность:** оценить влияние разных значений k (от 1 до 20) на точность.

Вариант 4: Ирисы Фишера (задача многоклассовой классификации)

- **Датасет:** `load_iris()` из `sklearn.datasets`
- **Цель:** Классификация сорта ириса по длине и ширине лепестка и чашелистика.
- **Особенность:** построить тепловую карту матрицы ошибок (`confusion_matrix + seaborn.heatmap`).

Вариант 5: Вина (задача многоклассовой классификации)

- **Датасет:** `load_wine()` из `sklearn.datasets`
- **Цель:** Определение сорта вина по химическим характеристикам.
- **Особенность:** сравнить качество k-NN при эвклидовой и манхэттенской метриках (параметр `metric='manhattan'`).

Вариант 6: Цифры (задача многоклассовой классификации)

- **Датасет:** `load_digits()` из `sklearn.datasets` (8×8 изображения рукописных цифр)
- **Цель:** распознавание рукописной цифры (0–9).
- **Особенность:** применить `StandardScaler` для масштабирования признаков (обучить `scaler` на обучающей выборке и преобразовать обе выборки). Обосновать необходимость масштабирования для k-NN.

Вариант 7: Физические упражнения Linnerud (задача регрессии с множеством целевых переменных)

- **Датасет:** `load_linnerud()` из `sklearn.datasets`
- **Цель:** прогнозирование количества подтягиваний, приседаний и прыжков по физическим показателям.
- **Особенность:** использовать `MultiOutputRegressor` с `LinearRegression` для предсказания всех трёх целевых переменных сразу.

Вариант 8: Diabetes с собственными преобразованиями (задача регрессии)

- **Датасет:** `load_diabetes()`
- **Цель:** сравнить линейную регрессию на исходных признаках и на признаках, возведённых в квадрат (полиномиальные признаки).
- **Особенность:** для создания полиномиальных признаков использовать `PolynomialFeatures(degree=2, include_bias=False)`.

Вариант 9: Анализ гамма-всплесков (задача бинарной классификации)

- **Датасет:** `fetch_openml(data_id=42733)` — датасет HIGGS
(требуется установка `pip install scikit-learn>=1.2` и интернет-соединения)
- **Цель:** Различение сигнала и фона в физике частиц.
- **Особенность:** Работа с большим объёмом данных (первые 10 000 строк), оценка времени предсказания для разного количества соседей.

Вариант 10: Квартиры в Сан-Франциско (задача регрессии) — Игрушечный датасет

Используйте датасет `fetch_california_housing()`.

- **Датасет:** `fetch_california_housing()`
- **Цель:** сравнить линейную регрессию и `KNeighborsRegressor` с разным `k`.
- **Особенность:** построить график зависимости метрики MSE от параметра `k` (от 1 до 30) для обоих алгоритмов и выбрать лучший.

Контрольные вопросы для самопроверки

1. Зачем нужен параметр `random_state` в функции `train_test_split()`? Что произойдёт, если его убрать?
2. Почему для k-NN критически важна нормализация признаков, а для линейной регрессии — нет?
3. Что такое переобучение (overfitting)? Как оно проявляется на графике зависимости точности от k?
4. В чём принципиальная разница между алгоритмами, которые строят модель (например, линейная регрессия) и ленивыми алгоритмами (k-NN)?
5. Какая метрика качества показывает долю объяснённой дисперсии? Каков диапазон её значений?
6. В каких случаях `accuracy` может быть обманчивой метрикой? Приведите пример.
7. В каком соотношении обычно делят данные на обучающую и тестовую выборки? Почему не стоит использовать тестовые данные для настройки гиперпараметров?
8. Что такое k в k-NN? Как выбор значения k влияет на границу между классами?
9. В чём разница между `predict()` и `predict_proba()` для классификатора?
10. Какая метрика лучше подходит для несбалансированных классов: `accuracy`, `precision`, `recall` или F1-score? Почему?

Дополнительные источники информации и литература

1. **Официальный учебник scikit-learn**
<https://scikit-learn.org/stable/tutorial/index.html>
2. **`train_test_split` — документация**
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
3. **Линейная регрессия в scikit-learn**
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
4. **`KNeighborsClassifier` — документация**
<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>
5. **Метрики качества sklearn.metrics**
https://scikit-learn.org/stable/modules/model_evaluation.html
6. **Реальные Python-проекты для начинающих**
Real Python: <https://realpython.com/train-test-split-python-data/>
7. **Лучшие датасеты для тренировки**
Kaggle Datasets: <https://www.kaggle.com/datasets>
8. **Книга «Python и анализ данных» (Уэс Маккинни)** — главы по scikit-learn.
9. **Интерактивный курс по машинному обучению на Stepik**
<https://stepik.org/course/4852>

Критерии оценивания (100-балльная шкала)

№	Критерий	Баллы	Комментарий
1.	Загрузка и первичный анализ данных	10	Загрузка датасета, вывод <code>.shape</code> , <code>.info()</code> , <code>.head()</code> , <code>.describe()</code>
2.	Подготовка данных (X, y, <code>train_test_split</code>)	10	Правильное выделение признаков и целевой переменной; обоснование <code>test_size</code>
3.	Обучение модели типа регрессии (линейная) или k-NN (классификация)	15	Выбор алгоритма в соответствии с вариантом, корректный <code>fit()</code>
4.	Предсказание на тестовой выборке	10	<code>predict()</code> на <code>X_test</code> , сохранение в переменную
5.	Оценка качества модели (не менее 2 метрик)	20	Регрессия: MSE/R ² /MAE; классификация: ассурасу, репорт или <code>confusion matrix</code>
6.	Попытка улучшения модели (выбор k, масштабирование, полиномиальные признаки)	15	Логичное обоснование изменений; повторная оценка и сравнение
7.	Интерпретация результатов	10	Содержательный вывод: какая модель лучше, почему, где ошибки
8.	Оформление кода и блокнота	10	Комментарии, логическая структура ячеек, наличие Markdown-пояснений
9.	Использование системы контроля версий Git (push на GitHub)	+5	Бонусные баллы за загрузку работы в удалённый репозиторий
	ИТОГО	100+5	

Шкала перевода в традиционную оценку:

- 90–100 (105 с бонусом) → **отлично**
- 75–89 → **хорошо**
- 60–74 → **удовлетворительно**
- Менее 60 → **неудовлетворительно** (задание требуется пересдать)